PFS-VSP-007 & PFS-VSP-006

Weekend Exam on For Loops - Answers

Each program includes the first test-case input. Replace that input to try the other test cases. All 30 test cases from the question paper have been verified.

1. Shopping Cart: Discount at the Right Time

Accumulate each price. Apply the discount once, after the full subtotal is known. The >= comparison includes exactly Rs. 2000.

Python solution

```
prices = [1200, 450, 350]

subtotal = 0
for price in prices:
    subtotal += price
discount = 0
if subtotal >= 2000:
    discount = subtotal * 0.10
print(f"Subtotal: {subtotal:.2f}")
print(f"Discount: {discount:.2f}")
print(f"Final Bill: {subtotal - discount:.2f}")
```

Output for the shown input

```
Subtotal: 2000.00
Discount: 200.00
Final Bill: 1800.00
```

2. Password Audit: Count and Decide

Each character belongs to exactly one of the four categories. A strong password must satisfy the length rule and every category rule.

Python solution

```
password = 'Python@1'

upper = lower = digits = special = 0
for ch in password:
    if 'A' <= ch <= 'Z':
        upper += 1
    elif 'a' <= ch <= 'z':
        lower += 1
    elif '0' <= ch <= '9':
        digits += 1
    else:
        special += 1
status = "WEAK"
if len(password) >= 8 and upper > 0 and lower > 0 and digits > 0 and special > 0:
    status = "STRONG"
print("Uppercase:", upper)
print("Lowercase:", lower)
print("Digits:", digits)
print("Special:", special)
print("Status:", status)
```

Output for the shown input

```
Uppercase: 1
Lowercase: 5
Digits: 1
Special: 1
Status: STRONG
```

3. Watch History: Ignore Skipped Entries

Increment the serial number only when a title is printed. continue skips the marker without leaving gaps in numbering.

Python solution

```
history = ['Leo', 'SKIP', 'Jailer', 'Leo']

count = 0
for title in history:
    if title == "SKIP":
        continue
    count += 1
    print(f"{count}. {title}")
print("Total Watched:", count)
```

Output for the shown input

```
1. Leo
2. Jailer
3. Leo
Total Watched: 3
```

4. Salary Report: Extremes and Threshold

Handle empty records first. Initialize the extremes from the first actual salary, then update totals, count and threshold count.

Python solution

```
salaries = {'Ravi': 50000, 'Teja': 60000, 'Ajay': 90000}

if len(salaries) == 0:
    print("No employee records")
else:
    highest = lowest = None
    total = count = eligible = 0
    for name in salaries:
        salary = salaries[name]
        if highest is None or salary > highest:
            highest = salary
        if lowest is None or salary < lowest:
            lowest = salary
        total += salary
        count += 1
        if salary >= 60000:
            eligible += 1
    print("Highest:", highest)
    print("Lowest:", lowest)
    print(f"Average: {total / count:.2f}")
    print("At Least 60000:", eligible)
```

Output for the shown input

```
Highest: 90000
Lowest: 50000
Average: 66666.67
At Least 60000: 2
```

5. Cricket Scorecard: Validate Every Ball

Validate each entry before using it. continue prevents an invalid value from affecting runs, boundaries or dot balls.

Python solution

```
runs = [4, 6, 1, 0, 2, 4, 0, 6]

total = boundaries = dots = invalid = 0
for value in runs:
    if value not in (0, 1, 2, 3, 4, 6):
        invalid += 1
        continue
    total += value
    if value == 4 or value == 6:
        boundaries += 1
    elif value == 0:
        dots += 1
print("Total Runs:", total)
print("Boundaries:", boundaries)
print("Dot Balls:", dots)
print("Invalid Entries:", invalid)
```

Output for the shown input

```
Total Runs: 23
Boundaries: 4
Dot Balls: 2
Invalid Entries: 0
```

6. Email Report: Exact Domain Match

The outer loop processes emails. The inner loop collects characters after @. Compare the complete extracted domain using ==.

Python solution

```
emails = ['ravi@codegnan.com', 'a@gmail.com', 'hr@codegnan.com']

count = 0
for email in emails:
    domain = ""
    after_at = False
    for ch in email:
        if after_at:
            domain += ch
        elif ch == "@":
            after_at = True
    print(domain)
    if domain == "codegnan.com":
        count += 1
print("Codegnan Emails:", count)
```

Output for the shown input

```
codegnan.com
gmail.com
codegnan.com
Codegnan Emails: 2
```

7. ATM PIN: Stop at the Correct Attempt

Count each processed PIN. Check success before stopping after the third attempt, so a correct third PIN succeeds. break ignores all later entries.

Python solution

```
attempts = ['1111', '2222', '1234']

used = 0
granted = False
for pin in attempts:
    used += 1
    if pin == "1234":
        granted = True
        break
    if used == 3:
        break
if granted:
    print("Access Granted")
else:
    print("Card Blocked")
print("Attempts Used:", used)
```

Output for the shown input

```
Access Granted
Attempts Used: 3
```

8. Guessing Game: Quit Without Using a Guess

Check 0 before counting or comparing it. Count every other processed guess, including the successful guess. break ends the game.

Python solution

```
guesses = [10, 40, 25, 99]

used = 0
for guess in guesses:
    if guess == 0:
        print("Game Cancelled")
        break
    used += 1
    if guess == 25:
        print("Correct Guess")
        break
    elif guess < 25:
        print("Too Low")
    else:
        print("Too High")
print("Guesses Used:", used)
```

Output for the shown input

```
Too Low
Too High
Correct Guess
Guesses Used: 3
```

9. Food Counter: Bill Until EXIT

Check EXIT first. Charge and count valid menu items; count unknown names separately. Repeated valid entries add another item and charge.

Python solution

```
orders = ['Burger', 'Tea', 'Fries', 'Burger', 'EXIT', 'Pizza']

menu = {"Burger": 80, "Pizza": 150, "Fries": 60}
items = bill = invalid = 0
for item in orders:
    if item == "EXIT":
        break
    if item in menu:
        items += 1
        bill += menu[item]
    else:
        invalid += 1
print("Items Ordered:", items)
print("Total Bill:", bill)
print("Invalid Orders:", invalid)
```

Output for the shown input

```
Items Ordered: 3
Total Bill: 220
Invalid Orders: 1
```

10. Quiz Game: Score, Lives and Two Stop Conditions

Update the score or lives for each answer. Stop as soon as either target is reached. Correct answers accumulate even when wrong answers occur between them.

Python solution

```
answers = ['Python', 'Java', 'Python', 'Python', 'C++']

lives = 3
score = used = 0
for answer in answers:
    used += 1
    if answer == "Python":
        score += 10
    else:
        lives -= 1
    if score == 30 or lives == 0:
        break
if score == 30:
    print("You Win")
else:
    print("Game Over")
print("Score:", score)
print("Lives Remaining:", lives)
print("Answers Used:", used)
```

Output for the shown input

```
You Win
Score: 30
Lives Remaining: 2
Answers Used: 4
```